

```
1) letters= 0
   For i in range length of goal:
       If state[i] != goal[i]:
           letters+= 1

   If letters= 0:
       Return letters

   Elif cost == 'steps':
       Return letters
```

For **steps**, it counts how many letters the state and the goal differ by. This is the lower bound, as the current state needs that remaining number of letters to make it to the goal state. It can't overestimate since it doesn't know whether the word will change to a word with no letters that match the goal state. At minimum, it will need to change the letters that don't match up, and since it isn't at the goal, it will need to change the letter count.

```
Elif cost == 'scrabble':
    total= 0
    For i in range length of goal:
        If state[i] != goal[i]:
            If goal[i] in "aeioulnstr":
                total= total + 1
            If goal[i] in "dg":
                total= total + 2
            If goal[i] in "bcmp":
                total= total + 3
            If goal[i] in "fhvwy":
                total= total + 4
            If goal[i] in "k":
                total= total + 5
            If goal[i] in "jx":
                total= total + 6
            If goal[i] in "qz":
                total= total + 10
    Return total
```

For **scrabble**, it is the lower bound because it finds out how many letters are different and the total cost of those missing letters. Similar to steps, it is a lower bound estimate of how many

letters still need to be changed to get to the goal state. It cannot overestimate since, again, it doesn't know whether or not it will turn into a word that has no letters of the goal word. At minimum, it will need to change the letters that don't match up, and since it isn't at the goal, it will need to change the letter count.

```

letters= 0
For i in range length of goal:
    If state[i] != goal[i]:
        letters+= 1
Elif cost == 'frequency':
    Return letters + 1 + rarity(goal)

```

For **frequency**, it is the lower-bound estimate because it counts how many letters need to change to get to the goal word, along with the rarity of the goal word, since once it gets to the goal word it will need to calculate its rarity for the cost. It doesn't overestimate because it doesn't know the next words or how many letter changes it will take to get to the goal word. The minimum is the number of letters that need to be changed, along with the rarity of the goal word.

2)

Word Pair	Cost Measure	UCS Cost	UCS Nodes Expanded	A* Cost	A* Nodes Expanded	Reduction
Why→Ask	Steps	5	773	5	16	98
Why→Ask	Scrabble	9	765	9	13	98
Why→Ask	Frequency	38.86	808	38.86	634	22
Bear→Duck	Steps	4	691	4	6	99
Bear→Duck	Scrabble	11	2981	11	6	99.8
Bear→Duck	Frequency	38.24	1084	38.24	302	72

Yes, UCS and A* always returned the same optimal cost. For Why→Ask, the largest percentage reduction by A* is Frequency. For Bear→Duck, the largest percentage reduction by A* is also Frequency. Frequency's heuristic was the least informative, as the reduction was smaller. The differences in the reduction are caused by the cost measure. With steps and scrabble, the letters determine the cost; however, for frequency, the word determines the cost.

3) For steps, yes, the reverse would have to be the same cost as the forward. This is because changing a letter is the cost, and you would be changing the same set of letters the same number of times. Meaning both costs would have to be equal to each other because the path is the same and the cost of that path doesn't change. For Scrabble, the reverse would also be the same as the forward since the path is the same as well. The cost determination is based on the letter, which is the same if the path is the same. In both cases, the letters essentially determine the cost; however, for frequency, the word itself determines the cost, which is why the reverse path might not be the same or have the same cost as the forward path. A counterexample would be dog and cat, dog $\rightarrow$ cat is around 23, whereas cat $\rightarrow$ dog is around 22.

4) GPT- 5.6 Luna: In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost

A good way to make h_0 rarity-aware is to use the minimum possible rarity cost needed to change the remaining incorrect letters.

Let:

- n = current word
- g = goal word
- $d(n, g)$ = number of positions where n and g differ (this is $h_0(n)$)
- Every action enters a new word w and costs $1 + \text{rarity}(w)$.

Proposed heuristic

Define

$$h(n) = d(n, g) + \sum_{i \in D(n, g)} r_i$$

where $D(n, g)$ is the set of positions where n differs from g , and

$r_i = \min\{\text{rarity}(w) : w \text{ is a legal word that can be entered while fixing position } i\}$

However, there is an important issue: a single intermediate word can fix multiple incorrect positions, so simply adding the minimum rarity for every position could count the same transition multiple times and therefore overestimate.

A safer admissible version is:

$$h(n) = h_0(n) + \min_{w \in P} \sum \text{rarity}(w)$$

where P ranges over all possible sequences of legal words that could be used to correct the remaining mismatched positions, with the restriction that each position is changed toward its goal value at most once.

In other words, the heuristic estimates:

1. At least $h_0(n)$ letter changes are necessary, and
2. For those necessary changes, we add the minimum rarity cost that could possibly be paid.

Why it is admissible

Every state that differs from the goal in k positions requires at least k actions, because one action changes only one letter.

Furthermore, every actual path must pay the rarity component of the words it enters. The heuristic uses the minimum possible rarity contribution among paths capable of making the required changes. Therefore,

$$h(n) \leq h^*(n),$$

where $h^*(n)$ is the true minimum remaining cost.

Thus it cannot overestimate.

Why it is more informative than h_0

Since rarity values are nonnegative,

Since rarity values are nonnegative,

$$h(n) \geq h_0(n).$$

And if there is at least one state for which every way of making the remaining changes requires entering a word with positive rarity, then

$$h(n) > h_0(n).$$

So it satisfies the assignment's definition of "more informative."

One caution: I would implement this using a formulation that finds the minimum rarity contribution over feasible remaining changes, rather than simply adding the minimum rarity for each mismatched position independently. The latter can double-count a single word's rarity and lose admissibility.

letter=0

Anika H

```
For i in range(length(n)):
    If n[i] != goal[i]:
        letter= letter + 1
```

```
min_rarity= 10000
```

```
For each word w in dictionary:
    If len(w) == len(n):
        change= 0
        For i in range(len(w)):
            If w[i] != n[i]:
                change+= 1

        If change ==1:
            If rarity(w) < min_rarity:
                min_rarity= rarity(w)
```

```
Return letter + min_rarity
```

It is admissible since it goes through the whole dictionary to find the word and see if it is the smallest cost. If it is, it becomes the new smallest cost, and it continues until it finds the smallest of them all.

After testing frequency, the amount for UCS is relatively the same; however, A* is back to being lower. For Why to Ask, it went down to 13 expanded nodes, but for Bear to Duck it is still the same: 302 nodes expanded. It might depend on the number of letters, but I think it is a good heuristic since one of the values went down.